

BOS: Bottleneck-Oriented Overlap Scheduling for Identical Parallel Machines

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan
leon28473787@gmail.com

Abstract—We study non-preemptive scheduling on identical parallel machines when every precedence edge permits a bounded amount of temporal overlap. For an edge (i, j) , task j may start after task i has processed for $\ell_{ij} = p_i - \delta_{ij}$ time units. This minimum start–start lag models partial-kit execution in multi-level Bill-of-Materials systems without reducing either task’s processing requirement.

We propose BOS-Greedy, an event-driven list scheduler that orders executable tasks by their maximum overlap-adjusted downstream path length. The score is computed by a reverse-topological dynamic program and the executable set is updated at release, activation, and completion events, including zero-lag activations at the current time stamp.

The analysis separates two models that were previously conflated. When all release times are zero, every work-conserving schedule in this model, including BOS-Greedy, satisfies $C_{\max} \leq W/m + (1 - 1/m)L_\delta$ and is therefore a $(2 - 1/m)$ -approximation. The proof constructs one schedule-dependent blocking chain and charges all non-full machine time to disjoint prefixes on that chain. With arbitrary release times, a release-aware path lower bound yields a safe 2-approximation.

Experiments on multi-level BOM instances show that BOS-Greedy is competitive with HEFT, consistently improves on FIFO, and reaches the optimum on the tested small exact instances. The guarantee is priority-independent; the experiments evaluate the additional value of the BOS dispatching priority.

Index Terms—Multi-level BOM scheduling, partial-kit activation, δ -overlap, minimum time lag, list scheduling.

I. INTRODUCTION

Scheduling in multi-level Bill-of-Materials (BOM) systems introduces challenges that are not captured by classical precedence-constrained scheduling models. In such environments, task dependencies are governed not only by logical precedence relations, but also by asynchronous material arrivals, partial-kit execution policies, and dynamically shifting bottlenecks.

Traditional models such as $P \parallel \text{prec} \parallel C_{\max}$ use finish–start precedence and usually assume that every task is available at time zero [1], [2]. In practice, an operation may start after each predecessor has produced the material needed for its first executable stage, even though those predecessors have not finished. We represent this rule by an edge-specific minimum start–start lag. The logical dependency remains an AND constraint, but its temporal activation occurs before predecessor completion.

Advanced Planning and Scheduling (APS) systems play a central role in modern manufacturing by coordinating material

availability, resource capacity, and precedence constraints. In practice, APS engines must repeatedly revise schedules in response to asynchronous material arrivals, bottleneck shifts, and partial-kit feasibility.

However, the commonly adopted practice of partial assembly introduces substantially greater scheduling complexity than assumed in traditional MRP frameworks [3]. When certain components arrive early, upstream operations may begin immediately, while critical components join only in later stages.

Thus, operation initiation is governed by a time-overlap activation condition: a task i may start only after each predecessor j has completed at least $p_j - \delta_{ji}$ units of processing, i.e.,

$$s_i \geq \max_{j \in \text{pred}(i)} (s_j + p_j - \delta_{ji}). \quad (1)$$

Actual start times are influenced by material releases, overlap allowances, the depth of the BOM hierarchy, and competition for machines. For a fixed input instance, the longest path in the precedence DAG is static. The resource-feasible critical chain, however, also depends on the machine-order edges created by the realized schedule and can therefore differ from that precedence-only path [4], [5].

Scheduling with precedence constraints on parallel machines is NP-hard [6]. List Scheduling provides a fundamental approximation guarantee of $(2 - 1/m)$ for such problems [7], forming a cornerstone of classical scheduling theory. Yet prior work shows that local greedy decisions can lead to timing anomalies in multiprocessor environments, where adding resources paradoxically increases completion time [7]. Furthermore, the literature on Resource-Constrained Project Scheduling (RCPS) indicates that, under resource contention, the effective completion time is governed by the resource-feasible critical chain, rather than the static longest path used by heuristics such as LPT [8]–[10]. Thus, the static longest-path perspective is insufficient in practical systems where bottlenecks are shaped by resource competition, material delays, and workload fluctuations [11], [12].

Approximation frameworks for shop scheduling—such as the ideal-schedule $\rightarrow$ random-delay $\rightarrow$ flattening paradigm—offer polylogarithmic guarantees [13], [14], but they fail to capture cross-level dependencies in multi-level BOMs or asynchronous material arrivals [15]. Similarly, AND/OR precedence models allow partial predecessor completion but do not explicitly address lead times, hierarchical depth, or delay

tolerance, making them unsuitable for multi-level manufacturing scenarios involving partial assembly [16]. These ingredients have individually appeared in precedence, time-lag, and resource-constrained scheduling models [16]–[18]. The question addressed here is narrower: can a simple event-driven list scheduler preserve a deterministic worst-case guarantee when an edge may activate after only a prefix of its predecessor has executed? We answer this question for identical machines and make the assumptions needed by the guarantee explicit.

In multi-level manufacturing processes, overall completion time is dictated by the dynamic behavior of the bottleneck chain rather than a static longest path. This behavior results from heterogeneous material arrivals, mid-process insertion of critical components, early operation initiation allowed by δ -tolerance, and shifts in bottlenecks caused by fluctuating machine loads [19]. Thus, future production scheduling and control must not rely solely on static rules; they must account for increasingly dynamic and uncertain manufacturing environments.

Although machine learning has been widely considered a promising enhancement to heuristic scheduling, recent surveys emphasize that such techniques cannot yet serve as full-scale replacements for complex decision-making algorithms [20]. Their performance depends heavily on specific data distributions or workload patterns and may degrade sharply under environmental shifts. Deep models also require extensive training data and costly offline exploration. Without approximation guarantees or feasibility assurances, learning-based scheduling can yield unpredictable or suboptimal outcomes in highly time-sensitive manufacturing settings—risking increased idle time, longer production cycles, or failure to meet delivery deadlines.

Furthermore, most learning-based scheduling or routing approaches lack interpretability, feasibility guarantees, and the theoretical performance bounds characteristic of classical algorithms. Therefore, learning-based approaches are better positioned as predictive auxiliary mechanisms rather than complete substitutes for structured scheduling theory.

These observations motivate a deterministic method that combines explicit overlap activation, an auditable dispatching rule, and a proof whose charging argument follows the causal path that actually delays completion.

Our contributions are summarized as follows:

- We instantiate an edge-wise minimum-lag scheduling model for partial-kit activation on identical parallel machines.
- We propose BOS-Greedy, an event-driven list scheduling algorithm that orders currently executable tasks by an overlap-adjusted downstream path metric.
- We give a complete proof chain: dynamic-program correctness, schedule feasibility, termination, implementation complexity, a single blocking-chain lemma, a $(2 - 1/m)$ guarantee when $r_i = 0$, and a 2-approximation for arbitrary release times.
- We use a schedule-induced augmented DAG to distinguish precedence bottlenecks from machine-order bottle-

necks, and evaluate BOS against HEFT, FIFO, and exact small-instance optima.

II. RELATED WORK

Multi-level manufacturing scheduling involves hierarchical precedence constraints, multi-machine resource competition, cross-level material arrivals, and partial-kit activation. These characteristics cannot be fully captured by a single classical scheduling model. This section reviews four major bodies of literature to identify the theoretical limitations faced when modeling multi-level BOM structures and δ -overlap activation.

A. Problem Setting: Multi-Level BOM, Hierarchical Precedence, and Dynamic Bottlenecks

Modern manufacturing systems commonly employ multi-level Bills of Materials (BOMs) to represent product structures. The parent–child relationships in a BOM naturally form a directed acyclic graph (DAG) [21], [22]. A BOM distinguishes between independent demand for finished goods and dependent demand for subcomponents, and its deep hierarchical structure determines the permissible order of operations across production stages.

However, multi-level BOM scheduling differs fundamentally from classical precedence-constrained parallel-machine models such as $P \parallel \text{prec} \parallel C_{\max}$. Although List Scheduling provides a $(2 - 1/m)$ approximation bound [7], the model assumes that all tasks are known at time zero and depend solely on logical precedence relations. It does not incorporate material availability, supply lead times, or asynchronous cross-level component arrivals, and therefore represents a flat and static precedence graph.

In contrast, dependencies in multi-level BOMs arise from *material causality*. Higher-level assemblies cannot begin until all subcomponents have been procured, processed, and physically delivered. Thus, start times are shaped by real-world supply delays, queueing effects, processing load variations, and heterogeneous arrival patterns. This leads to a time-sensitive DAG whose behavior is driven by dynamic material availability.

Moreover, bottlenecks in multi-stage production may shift with material delays, machine congestion, or temporary queue buildup [23], [24]. A precedence-only longest path is fixed for one input instance, whereas the resource-feasible critical chain also depends on the machine order induced by the schedule.

Overall, multi-level BOM scheduling is difficult because:

- operation start times depend on actual material arrivals rather than predetermined release constraints;
- delays in any component may propagate upward across layers;
- bottleneck locations evolve over time due to material flow variability and resource contention.

These characteristics extend the standard $P \parallel \text{prec} \parallel C_{\max}$ model with release dates and edge-wise minimum start–start lags.

B. Partial-Kit Activation, δ -Overlap, and Asynchronous Material Arrivals

A second key distinction from classical scheduling models is that many manufacturing systems allow operations to begin before all predecessors are completed. Industries such as semiconductor packaging, automotive electronics, and assembly manufacturing often adopt *partial-kit* policies [25]: once a subset of required components arrives, upstream operations start immediately to reduce wait time caused by late-arriving parts. This deviates from the traditional AND-precedence assumption that all predecessors must be completed.

This work formalizes such behavior using a tolerance parameter δ , representing the allowable degree of overlap between missing dependencies and operation initiation. Under δ -overlap activation, an operation may begin as long as the missing components do not block the executable portion. This is neither classical AND-precedence nor OR-precedence [16], but an intermediate dependency that allows limited overlap. The value of δ directly influences the feasible scheduling space.

Material arrivals in multi-level BOM systems are typically asynchronous. Heterogeneous arrival times cause some stages to begin early while others must wait, producing a hybrid execution pattern of “partially overlapping” and “partially blocked” operations.

Classical finish–start precedence, RCPSP [8], and AND/OR models [16] encode different activation semantics. The present δ -overlap rule is most directly viewed as an edge-wise minimum start–start lag. Classical approximation bounds cannot simply be quoted for this model; the required idle-time charging statement must be re-proved.

C. Dynamic Bottlenecks and Critical-Chain Deviation

In multi-stage manufacturing, the realized makespan may be governed by resource order as well as precedence. Temporary overloads, queue buildup, machine delays, and late materials can change which realized chain is longest [4]. A static precedence DAG does not contain machine order, but a schedule-induced augmented DAG can represent both types of constraint.

D. Classical Deterministic Scheduling Algorithms

1) *Method Categories*: Deterministic approximation algorithms for precedence scheduling can be grouped into five major categories:

a) *Greedy / List Scheduling*: Simple, practical, and provides the $(2 - 1/m)$ bound for zero-release finish–start precedence [7]. Event-driven implementations can support changing ready sets, but release dates and overlap lags require a separate analysis.

b) *HEFT (Heterogeneous Earliest Finish Time)*: A widely used DAG scheduler for heterogeneous processors [26]. Limitations include:

- all task information known at time zero,
- strict AND-precedence,
- static DAG structure,

- no support for δ -overlap or dynamic bottlenecks.

c) *LP Rounding*: Offers strong worst-case guarantees such as 2-approx or PTAS [13], [27], but assumes all tasks can be modeled in a fixed precedence graph at time zero—an assumption that may fail in online BOM settings.

d) *Primal–Dual / Dual-Fitting*: Effective for certain constrained settings but depends on static feasible domains [28].

e) *Local Search*: Heuristically strong but lacks theoretical guarantees and assumes fixed topology [29].

2) *Representative Approximation Ratios*:

- Graham List Scheduling: $(2 - 1/m)$
- LP-based: 2-approx or PTAS
- Shmoys–Stein–Wein: polylogarithmic approximations
- Potts & Van Wassenhove: 2–3 approximation for batching and related settings

3) *Limitations of Classical Methods*: The classical bounds cited above do not automatically cover all of:

- δ -overlap activation,
- asynchronous material arrival,
- dynamic bottleneck movement,
- cross-level delay propagation,
- multi-level BOM causal structures.

Thus, their assumptions must be checked before they are transferred to the partial-kit setting studied here.

E. Machine-Learning-Based Scheduling

Recent ML-based scheduling approaches include reinforcement learning, supervised dispatching models, and learning-to-schedule frameworks [30]–[32]. Their guarantees depend on the model and implementation; common risks include:

- no deterministic approximation ratio,
- degradation under workload shift,
- feasibility requiring a separate enforcement layer, and
- limited interpretability of individual dispatch decisions.

These limitations motivate retaining an auditable combinatorial scheduler, potentially with ML used as an auxiliary predictor.

F. Positioning of BOS: Contributions Over Prior Work

The Bottleneck-Oriented Overlap Scheduling (BOS) method addresses the limitations identified above by providing:

- 1) a deterministic, interpretable scheduling rule built on analyzable list-scheduling structure;
- 2) an overlap-adjusted bottom-level metric b_i ;
- 3) native support for multi-level BOM dependencies and material causal structure;
- 4) explicit modeling of δ -overlap activation;
- 5) event-driven release, activation, and completion updates;
- 6) a balance between theoretical analyzability and practical industrial feasibility.

BOS combines multi-level BOM structure, δ -overlap activation, and a deterministic list scheduler. Its realized resource bottleneck is diagnosed after scheduling through an augmented DAG rather than being silently conflated with the static precedence score.

TABLE I
GAP ANALYSIS BETWEEN EXISTING SCHEDULING APPROACHES AND BOS-GREEDY.

Category	Existing methods	Limitations	BOS-Greedy
Scheduling paradigm	Static-priority list scheduling	Priority may ignore overlap and release events	Event-driven readiness with overlap-adjusted priority
Precedence handling	Finish-start precedence constraints	No early successor activation	Minimum start-start lags for controlled overlap
Theoretical grounding	Classical Graham analysis	Does not directly prove the lagged model	Single-chain charging proof with explicit release-time scope

III. MODEL AND PRELIMINARIES

A. Scheduling Model

Let $G = (V, E)$ be a directed acyclic graph. Each task $i \in V$ has a processing time $p_i > 0$ and a finite release time $r_i \geq 0$. There are m identical machines; each task uses one machine, runs non-preemptively, and is assigned a start time s_i and a machine $\mu_i \in \{1, \dots, m\}$.

For every edge $(i, j) \in E$, let

$$0 \leq \delta_{ij} \leq p_i, \quad \ell_{ij} = p_i - \delta_{ij} \in [0, p_i]. \quad (2)$$

The overlap allowance δ_{ij} does not shorten either task. It only reduces the predecessor prefix that must elapse before j may start. A schedule is feasible if

$$s_i \geq r_i \quad \forall i \in V, \quad (3)$$

$$s_j \geq s_i + \ell_{ij} \quad \forall (i, j) \in E, \quad (4)$$

and tasks assigned to the same machine have disjoint execution intervals:

$$[s_i, s_i + p_i] \cap [s_j, s_j + p_j] = \emptyset \quad \text{whenever } \mu_i = \mu_j. \quad (5)$$

The objective is to minimize

$$C_{\max} = \max_{i \in V} (s_i + p_i). \quad (6)$$

Thus, $\delta_{ij} = 0$ recovers finish-start precedence, whereas $\delta_{ij} = p_i$ permits simultaneous starts on different machines.

B. Overlap-Adjusted Paths and Lower Bounds

For a directed path $P = (v_1, \dots, v_k)$, define its effective length by

$$\begin{aligned} Q_\delta(P) &= p_{v_k} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} \\ &= \sum_{h=1}^k p_{v_h} - \sum_{h=1}^{k-1} \delta_{v_h v_{h+1}}. \end{aligned} \quad (7)$$

Singleton paths are allowed, so $Q_\delta((i)) = p_i$. Let

$$L_\delta = \max_P Q_\delta(P). \quad (8)$$

For arbitrary release times, also define

$$Q_{r,\delta}(P) = r_{v_1} + Q_\delta(P), \quad L_{r,\delta} = \max_P Q_{r,\delta}(P). \quad (9)$$

Lemma 1 (Workload and path lower bounds). *Let $W = \sum_{i \in V} p_i$. Every feasible schedule satisfies*

$$C_{\max}^{\text{OPT}} \geq \max \left\{ \frac{W}{m}, L_{r,\delta} \right\}. \quad (10)$$

When all $r_i = 0$, this becomes $C_{\max}^{\text{OPT}} \geq \max\{W/m, L_\delta\}$.

Proof. The m machines provide at most $mC_{\max}$ units of processing capacity, hence $C_{\max} \geq W/m$. For any path $P = (v_1, \dots, v_k)$, repeatedly applying (3)–(4) gives

$$s_{v_k} + p_{v_k} \geq r_{v_1} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} + p_{v_k} = Q_{r,\delta}(P).$$

Maximizing over P proves the second lower bound. $\square$

C. BOS Bottom-Level Priority

For every task i , define

$$b_i = \max \left\{ p_i, \max_{(i,j) \in E} (\ell_{ij} + b_j) \right\}, \quad (11)$$

where the inner maximum is omitted for a sink. The explicit term p_i is necessary: a highly overlapped outgoing edge may make every continuation shorter than the singleton path containing i alone.

Lemma 2 (Correctness of the bottom-level dynamic program). *For every task i ,*

$$b_i = \max_{P: P \text{ starts at } i} Q_\delta(P). \quad (12)$$

Consequently, $\max_i b_i = L_\delta$, and all scores can be computed in $O(|V| + |E|)$ time by scanning a reverse topological order.

Proof. Proceed by reverse-topological induction. For a sink i , the only path starting at i is (i) , so both sides equal p_i . Suppose the claim holds for every successor j of i . A path starting at i is either the singleton (i) or consists of an edge (i, j) followed by a path starting at j . Its best length is therefore

$$\max \left\{ p_i, \max_{(i,j) \in E} \left(\ell_{ij} + \max_{P: P \text{ starts at } j} Q_\delta(P) \right) \right\},$$

which equals (11) by the induction hypothesis. Each vertex and edge is inspected once. $\square$

For a fixed offline instance, b_i is a static score. BOS is event-driven because the executable set changes over time; merely recomputing (11) on unchanged data would not make the score dynamic. Online changes to p , r , δ , or G may trigger incremental recomputation, but that extension is outside the guarantees proved below.

IV. BOS-GREEDY ALGORITHM AND CORRECTNESS

A. State and Executable Set

At time t , partition the tasks into the not-started, running, and completed sets $N(t)$, $A(t)$, and $C(t)$. Start times are defined for tasks in $A(t) \cup C(t)$. After all events stamped t have been processed, the executable set is

$$R(t) = \{i \in N(t) : t \geq r_i, j \in A(t) \cup C(t) \forall j \in \text{pred}(i), t \geq s_j + \ell_{ji} \forall j \in \text{pred}(i)\}. \quad (13)$$

BOS selects a task of maximum b_i from $R(t)$; a fixed task index breaks ties. The scheduler is work-conserving: after processing all same-time activations, it never leaves a machine idle while $R(t)$ is nonempty.

B. Complete Event-Driven Procedure

Three event types are required: a release at r_i , an edge activation at $s_i + \ell_{ij}$, and a completion at $s_i + p_i$. Starting a task can generate zero-lag activations at the current time. These events are processed to closure before time advances.

Algorithm 1 Event-Driven BOS-Greedy

Input: DAG $G = (V, E)$, p , r , δ , and m

Output: start times s_i and machine assignments μ_i

```

1: Compute  $\ell_{ij}$  and all  $b_i$  in reverse topological order
2:  $t \leftarrow 0$ ;  $N \leftarrow V$ ;  $A, C \leftarrow \emptyset$ 
3: Initialize the event queue with every release event
4: while  $C \neq V$  do
5:   Process every release, activation, and completion at time  $t$ 
6:   Insert every newly executable task into a max-heap keyed by  $b_i$ 
7:   while an idle machine and an executable task both exist do
8:     Remove a maximum-key task  $i$  from the ready heap
9:     Start  $i$ : set  $s_i \leftarrow t$ , assign an idle machine
10:    Move  $i$  from  $N$  to  $A$ ; enqueue completion at  $t + p_i$ 
11:    Enqueue  $(i, j)$  activation at  $t + \ell_{ij}$  for each  $(i, j) \in E$ 
12:    Process all newly generated events stamped  $t$ 
13:    Insert every task made executable at  $t$  into the ready heap
14:   end while
15:   if  $C \neq V$  then
16:     Advance  $t$  to the smallest future event time
17:   end if
18: end while
```

An implementation maintains, for every task, whether its release has occurred and how many incoming edges have activated. Thus line 5 does not rescan the entire graph. Completion events both free a machine and move a task from A to C .

Theorem 3 (Feasibility and single execution). *Algorithm 1 starts every task exactly once and returns a feasible schedule satisfying (3)–(5).*

Proof. A task can enter the ready heap only while it belongs to N . At its start it is removed from N , so it cannot be inserted or started a second time. A task is dispatched only after its release event and all incoming activation events have occurred. Its recorded start time therefore satisfies (3) and (4). Dispatch occurs only to an idle machine, and that machine remains

occupied until the task's completion event, proving (5). The zero-lag closure permits a successor to start at the same time as its predecessor, but only after the predecessor's start has been recorded, so the same argument still applies. $\square$

Lemma 4 (No deadlock and termination). *Algorithm 1 terminates after finitely many events and, together with Theorem 3, starts every task exactly once.*

Proof. Every start removes one task from N , so there are at most $|V|$ starts. Suppose $N \neq \emptyset$. If all machines are busy, a running task has a finite next completion time. Otherwise take a source v in the induced DAG $G[N]$. Every predecessor of v has already started. If v is not executable now, then either its finite release time or one of its finite edge-activation times is a future event; if neither condition holds, $v \in R(t)$ and it is dispatched. Same-time events are exhausted before advancing, so every time advance is strict. There are at most $|V|$ releases, $|E|$ activations, and $|V|$ completions. Hence no deadlock or infinite event sequence is possible. $\square$

Proposition 5 (Implementation complexity). *With adjacency lists, activation counters, an event heap, and a ready heap, BOS-Greedy runs in*

$$O((|V| + |E|) \log(|V| + |E|)) \quad (14)$$

time and uses $O(|V| + |E|)$ space.

Proof. Topological sorting and computing b_i cost $O(|V| + |E|)$. Each task generates one release, one start, and one completion, and each edge generates one activation. Each event is inserted into and removed from a heap a constant number of times. Each task enters and leaves the ready heap once. The stored graph, counters, heaps, and task state are all linear in $|V| + |E|$. $\square$

V. APPROXIMATION ANALYSIS

The following proof uses only work conservation, not the particular ordering by b_i . It therefore certifies BOS-Greedy but also applies to any work-conserving list scheduler under the same model. The BOS priority is evaluated empirically rather than credited with a bound that the proof does not use.

For a completed schedule S , let $q(t)$ be the number of busy machines, let

$$\mathcal{T}_{\text{nf}} = \{t \in [0, C_{\max}) : q(t) < m\}, \quad (15)$$

and define total idle machine-time as

$$I = \int_0^{C_{\max}} (m - q(t)) dt. \quad (16)$$

The integral is only accounting notation; the schedule changes at a finite set of event times, so it is exactly the sum over the resulting constant-state intervals.

For a task v in the realized schedule, define its eligibility time

$$a(v) = \max(\{r_v\} \cup \{s_u + \ell_{uv} : (u, v) \in E\}). \quad (17)$$

Clearly $a(v) \leq s_v$. Choose a last-finishing task v_k . Working backward from a current task v_h , stop if the release term r_{v_h} attains $a(v_h)$, choosing it in a tie. Otherwise choose a predecessor v_{h-1} satisfying

$$a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h} \quad (18)$$

and continue. Acyclicity makes this process terminate, producing a single directed path $P = (v_1, \dots, v_k)$. Put $\rho = r_{v_1}$ and $Q = Q_\delta(P)$.

Lemma 6 (Single blocking-chain cover). *Up to the zero-measure event instants, every non-full time belongs to*

$$\mathcal{T}_{\text{nf}} \subseteq [0, \rho) \cup \bigcup_{h=1}^{k-1} [s_{v_h}, s_{v_h} + \ell_{v_h v_{h+1}}) \cup [s_{v_k}, s_{v_k} + p_{v_k}). \quad (19)$$

The intervals after $[0, \rho)$ are pairwise disjoint, have total length Q , and each contains a running chain task. Consequently,

$$I \leq m\rho + (m-1)Q. \quad (20)$$

Proof. Take $t \in \mathcal{T}_{\text{nf}}$. If $t \geq s_{v_k}$, then t precedes the makespan and therefore lies in the execution interval of v_k . Otherwise $t < s_{v_k}$. If $t \geq a(v_k)$, then v_k would be executable while a machine is idle, contradicting work conservation. Hence $t < a(v_k)$.

If the chain step into v_h was chosen through predecessor v_{h-1} , then $t < a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h}$. Either $t \geq s_{v_{h-1}}$, in which case t lies in that predecessor prefix, or $t < s_{v_{h-1}}$, in which case the same argument is applied one step earlier. If the recursion reaches v_1 without covering t , then $t < s_{v_1}$ and work conservation gives $t < a(v_1) = r_{v_1} = \rho$. This proves (19).

For consecutive chain tasks, $s_{v_{h+1}} \geq s_{v_h} + \ell_{v_h v_{h+1}}$, so the charged prefixes are ordered and disjoint; the final execution interval begins no earlier than the last prefix ends. Their total length is $\sum_{h < k} \ell_{v_h v_{h+1}} + p_{v_k} = Q$. Since $\ell_{v_h v_{h+1}} \leq p_{v_h}$, task v_h is running throughout its charged prefix. Thus at most $m-1$ machines are idle on every charged time after ρ , while at most m are idle before ρ . Integrating these two bounds proves (20). $\square$

A. Zero Release Times

Theorem 7 (Overlap-aware Graham bound). *If $r_i = 0$ for all tasks, BOS-Greedy satisfies*

$$C_{\text{max}}^{\text{BOS}} \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta. \quad (21)$$

Proof. In Lemma 6, $\rho = 0$ and $Q \leq L_\delta$, so $I \leq (m-1)L_\delta$. Every unit of the time horizon contributes either busy or idle machine-time; hence

$$mC_{\text{max}}^{\text{BOS}} = W + I. \quad (22)$$

Substituting the idle bound into (22) and dividing by m proves (21). $\square$

Corollary 8 $((2 - 1/m)$ approximation). *If all release times are zero, BOS-Greedy is a $(2 - 1/m)$ -approximation.*

Proof. By Lemma 1, both W/m and L_δ are at most $C_{\text{max}}^{\text{OPT}}$. Applying these two inequalities to (21) yields

$$C_{\text{max}}^{\text{BOS}} \leq \left(2 - \frac{1}{m}\right) C_{\text{max}}^{\text{OPT}}. \quad \square$$

B. Arbitrary Release Times

The sharper zero-release formula cannot hold unchanged when release times are present. For example, a single unit-length task released at time R finishes at $R + 1$, whereas a bound containing only W/m and L_δ is independent of R .

Theorem 9 (Release-aware guarantee). *For arbitrary finite release times, BOS-Greedy satisfies*

$$C_{\text{max}}^{\text{BOS}} \leq \frac{W}{m} + L_{r,\delta} \leq 2C_{\text{max}}^{\text{OPT}}. \quad (23)$$

Proof. For the chain of Lemma 6, capacity accounting and (20) give

$$\begin{aligned} C_{\text{max}}^{\text{BOS}} &= \frac{W + I}{m} \\ &\leq \frac{W}{m} + \rho + \left(1 - \frac{1}{m}\right) Q \\ &\leq \frac{W}{m} + \rho + Q. \end{aligned}$$

Because P starts at v_1 , $\rho + Q = Q_{r,\delta}(P) \leq L_{r,\delta}$. The first inequality in (23) follows. Lemma 1 states that both W/m and $L_{r,\delta}$ are lower bounds on the optimum, which proves the factor of two. $\square$

C. Scope of the Guarantee

The proof assumes identical machines, non-preemptive tasks, a fixed DAG, fixed nonnegative lags satisfying $\ell_{ij} \leq p_i$, and work-conserving dispatch. It does not prove that the BOS priority dominates every other priority. Heterogeneous machines, uncertain or time-varying processing data, setup times, and online graph changes require separate analyses.

VI. SCHEDULE-INDUCED BOTTLENECK DIAGNOSIS

The precedence DAG alone cannot identify delay caused by competition for a machine. Given a feasible schedule S , add a virtual source 0 and, for each pair of consecutive tasks (i, j) on the same machine, a machine-order edge. The weighted augmented graph is

$$G_S^+ = (V \cup \{0\}, E_r \cup E \cup E_M), \quad (24)$$

where $(0, i) \in E_r$ has weight r_i , $(i, j) \in E$ has weight ℓ_{ij} , and $(i, j) \in E_M$ has weight p_i . Every edge (u, v) encodes $s_v \geq s_u + w_{uv}$ with $s_0 = 0$.

Lemma 10 (Augmented-DAG certificate). *For every feasible schedule, G_S^+ is acyclic. Its longest-path distances from 0 give*

the earliest feasible start times for the fixed machine order. An edge slack

$$\sigma_{uv} = s_v - s_u - w_{uv} \geq 0 \quad (25)$$

certifies whether a precedence or machine-order constraint is tight.

Proof. The virtual source has no incoming edge, and the original precedence graph is acyclic. Hence any directed cycle in G_S^+ would contain a machine-order edge of positive weight p_i . Summing $s_v \geq s_u + w_{uv}$ around that cycle would yield $s_v \geq s_v + \sum w_{uv} > s_v$, a contradiction. Since the graph is a DAG, the recurrence $d_v = \max_{(u,v)}(d_u + w_{uv})$ in topological order gives the earliest start compatible with all fixed edges. The slack formula follows directly from feasibility. $\square$

This representation supports a DAG analyzer that validates a schedule, extracts the realized critical chain, reports zero-slack constraints, and classifies a bottleneck as precedence-driven or machine-driven. The graph is schedule-induced: it explains how the bottleneck changes with dispatch decisions without incorrectly claiming that the fixed score b_i changes by itself.

VII. EXPERIMENTAL DESIGN

A. Experimental Objectives

The goal of this study is not merely to demonstrate that BOS achieves lower makespan than existing heuristics, but to systematically evaluate its behavior from three complementary perspectives:

- **Theory alignment:** To verify whether the empirical performance of BOS is consistent with the theoretical lower bound derived from workload and overlap-adjusted critical chains.
- **Mechanism comparison:** To analyze how BOS differs from static-priority schedulers such as HEFT in terms of decision dynamics, particularly under overlap-aware precedence constraints.
- **Robustness across structures:** To evaluate the performance of BOS across diverse DAG structures, machine configurations, and overlap conditions, ensuring that results are not restricted to specific handcrafted scenarios.

B. Scheduling Models and Baselines

We evaluate BOS against two representative scheduling policies:

- **HEFT (Heterogeneous Earliest Finish Time):** A widely used static-priority list scheduling algorithm based on upward rank.
- **FIFO (First-In-First-Out):** A simple baseline that schedules tasks in the order of readiness.

All schedulers operate under the same execution model:

- Parallel identical machines
- DAG precedence constraints
- Edge-wise overlap allowance δ_{ij}
- Event-driven simulation (release, activation, and completion)

C. Instance Generation

To ensure comprehensive evaluation, we construct three types of instance families.

1) *Neutral Benchmark Families:* We generate synthetic DAGs with controlled structural properties, including:

- **Chain-heavy:** deep dependency chains
- **Fork-join:** parallel branches with synchronization points
- **Merge-heavy:** multiple predecessors per task
- **Overlap-rich:** high density of overlap allowances

Each instance is parameterized by:

- Number of tasks (small / medium / large)
- Machine count $m \in \{2, 4, 8\}$
- Overlap density and magnitude
- Processing time distributions
- Release-time policy; the retained 20260323 benchmark plots use $r_i = 0$ for every task

This design avoids bias toward any specific scheduling strategy.

2) *Mechanism-Oriented Cases:* In addition to neutral benchmarks, we construct controlled instances to highlight scheduling behavior under:

- Bottleneck shifts
- Overlap-induced precedence relaxation
- Late-stage critical task emergence

These cases are used for qualitative analysis rather than aggregate performance comparison.

3) *Small Exact Instances:* For small problem sizes, we compute optimal schedules using a branch-and-bound solver.

This enables direct evaluation of:

$$\text{gap}_{OPT} = \frac{C_{\max} - C^*}{C^*} \quad (26)$$

providing ground-truth validation of algorithm performance.

D. Performance Metrics

1) Makespan:

$$C_{\max} \quad (27)$$

The total completion time of all tasks.

2) *Lower Bound Gap:* We compute the release-aware theoretical lower bound:

$$LB = \max\left(\frac{W}{m}, L_{r,\delta}\right). \quad (28)$$

When every $r_i = 0$, this reduces to $\max\{W/m, L_\delta\}$, which is the specialization used by the reported 20260323 benchmark plots. Any future experiment with nonzero releases must instead use $L_{r,\delta}$; a late release can dominate the makespan even when processing times are small.

We report:

$$\text{gap}_{LB} = \frac{C_{\max} - LB}{LB} \quad (29)$$

3) *Optimality Gap*: For instances where the optimal solution is available:

$$\text{gap}_{OPT} = \frac{C_{\max} - C^*}{C^*} \quad (30)$$

4) *Decision Disagreement Rate*: To quantify behavioral differences between schedulers, we measure:

$$\text{disagreement}(A, B) = \frac{\# \text{ of differing decisions}}{\text{total decisions}} \quad (31)$$

This metric captures how frequently two algorithms select different tasks under identical system states.

5) *Idle Time Decomposition*: Idle periods are categorized into:

- Precedence-constrained blocking
- Release-time blocking
- Mixed blocking

This allows analysis of scheduling inefficiencies beyond makespan.

E. Experimental Protocol

For each configuration:

- Multiple random instances are generated
- Each scheduler is run independently
- Results are averaged over all instances

We report:

- Mean makespan
- Mean lower-bound gap
- Mean optimality gap (if available)
- Mean decision disagreement rate

All experiments are conducted using the same event-driven simulation framework to ensure consistency.

F. Implementation Details

The simulator is implemented in Python and follows an event-driven design:

- Release, edge-activation, and completion events update task state
- The fixed BOS scores b_i are computed once per offline instance
- A ready heap selects the maximum- b_i executable task
- Same-time zero-lag activations are processed to closure

For small instances, optimal schedules are computed using a branch-and-bound solver with pruning based on partial schedule bounds.

VIII. RESULTS AND ANALYSIS

A. Overall Performance Trends

This section analyzes the empirical behavior of BOS, HEFT, and FIFO under the benchmark settings in Section VII. The results show three consistent patterns. First, BOS and HEFT achieve highly comparable makespan across most instance groups. Second, FIFO is consistently weaker than both BOS and HEFT, especially in medium and large structured instances. Third, the absolute performance gap between BOS

and HEFT is generally small, indicating that both methods operate in a regime close to the theoretical lower bound.

Figure 1 visualizes the relative makespan advantage of BOS over HEFT across all benchmark groups. The horizontal axis corresponds to different combinations of DAG family, instance size, and machine count, while the vertical axis reports the percentage improvement of BOS over HEFT. Most bars remain close to zero, showing that BOS does not consistently dominate HEFT in absolute makespan. However, this should not be interpreted as a weakness of BOS. Rather, it suggests that HEFT itself is already a strong baseline, and that the feasible improvement space is limited when the schedules produced by both methods are already near the theoretical lower bound.

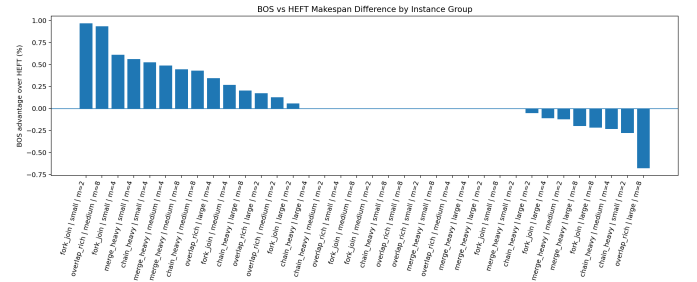


Fig. 1. Relative makespan advantage of BOS over HEFT across all benchmark groups. Positive values indicate that BOS achieves lower makespan, while negative values indicate that HEFT performs slightly better. Most differences remain close to zero, suggesting that both methods operate in a near-optimal regime.

To better understand whether the benchmark still distinguishes meaningful scheduling behavior, Figure 2 compares FIFO against BOS. Unlike the BOS–HEFT comparison, FIFO exhibits a clear penalty in many instance groups. This indicates that simple ready-order scheduling is insufficient under overlap-aware precedence constraints. In other words, the benchmark is not too easy; rather, it is sufficiently discriminative to separate structure-aware schedulers from naive dispatching rules.

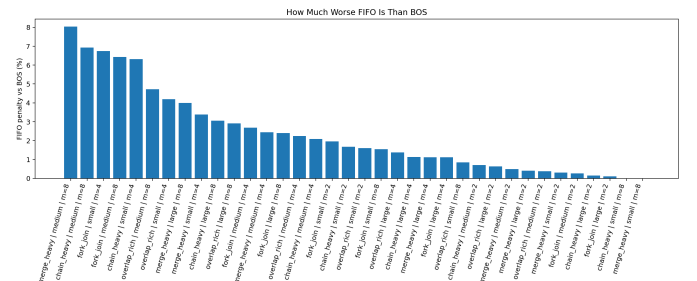


Fig. 2. Relative makespan penalty of FIFO compared with BOS. Larger positive values indicate that FIFO produces longer schedules. The results show that naive ready-order scheduling is consistently weaker than structure-aware methods.

B. Performance Relative to the Lower Bound

To evaluate schedule quality beyond raw makespan, we compare each algorithm against the theoretical lower bound

$$LB = \max\left(\frac{W}{m}, L_{r,\delta}\right).$$

Since this bound combines both workload-based and overlap-adjusted critical-chain constraints, it provides a theory-aligned reference point for empirical evaluation.

Figure 3 shows the difference between the lower-bound gap of HEFT and that of BOS. Positive values indicate cases where BOS is closer to the lower bound than HEFT, while negative values indicate the opposite. The figure shows that the gap difference is usually small, but BOS often remains at least as close to the lower bound as HEFT, and in several groups it is slightly tighter. This observation is important because it connects the empirical results with the approximation-oriented analysis developed earlier. Even when BOS does not substantially reduce raw makespan relative to HEFT, it still behaves competitively from the perspective of theoretical schedule tightness.

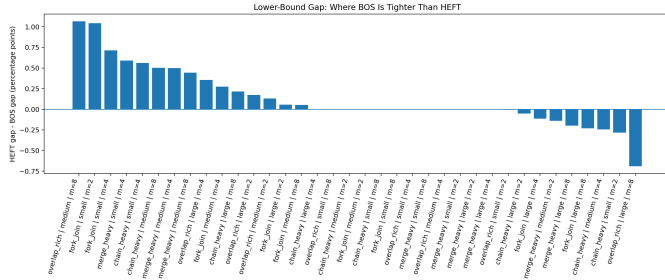


Fig. 3. Difference in lower-bound gap between HEFT and BOS. Positive values indicate that BOS is closer to the theoretical lower bound than HEFT. The small but nontrivial differences suggest that BOS remains competitive in terms of theory-aligned schedule quality.

More importantly, the small gap-to-bound values observed for both BOS and HEFT imply that the scheduling space is already tight. Therefore, the absence of a large makespan advantage for BOS should not be read as a lack of contribution. Instead, it suggests that under overlap-aware precedence constraints, multiple strong scheduling mechanisms may converge to similarly high-quality schedules.

C. Decision-Process Divergence Between BOS and HEFT

Although BOS and HEFT often produce similar makespan, their internal decision processes differ substantially. To quantify this difference, we introduced the decision disagreement rate, which measures how often two schedulers choose different tasks under the same system state.

Figure 4 shows the disagreement rate between BOS and HEFT across all instance groups. Several groups, especially in overlap-rich and large-scale settings, exhibit high disagreement rates. This means that BOS and HEFT frequently make different dispatching choices even though their final makespan remains close. Such behavior indicates that BOS

is not a trivial re-labeling of HEFT. Rather, BOS combines event-driven readiness with an overlap-adjusted bottom-level priority, whereas the HEFT baseline follows its static upward-rank priority.

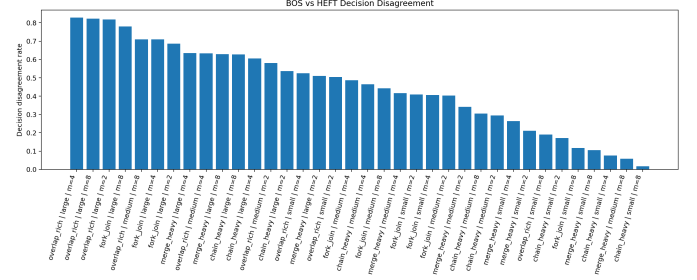


Fig. 4. Decision disagreement rate between BOS and HEFT. High values indicate that the two schedulers frequently choose different tasks under comparable states. This confirms that BOS and HEFT rely on fundamentally different dispatching mechanisms despite often producing similar makespan.

This result is one of the central findings of the study. A large decision discrepancy does not necessarily translate into a large makespan discrepancy. This suggests that under overlap-aware precedence constraints, multiple distinct scheduling strategies may converge to near-optimal solutions. From a research perspective, this strengthens the significance of BOS: its value lies not only in numerical performance, but also in providing an alternative and theoretically grounded scheduling framework.

D. Exact Validation on Small Instances

To further validate solution quality, we computed optimal schedules for small instances using the branch-and-bound solver described in Section VII. Figure 5 reports the average gap to optimality for BOS, HEFT, and FIFO on the exact benchmark families.

The results show that BOS reaches zero optimality gap on all tested small families. HEFT also performs strongly, but is not uniformly optimal in every family. FIFO, by contrast, remains consistently worse. These findings provide two important conclusions. First, BOS is not merely a heuristic that performs well on average; on small exact instances, it can recover the optimal schedule. Second, the strong behavior of both BOS and HEFT again confirms that the main empirical challenge is not beating weak baselines, but understanding how different high-quality scheduling mechanisms behave under structured overlap constraints.

E. Discussion

Taken together, the results support the following interpretation.

First, BOS is competitive with HEFT in terms of makespan, but does not consistently dominate it in neutral benchmark settings. This is a realistic and theoretically coherent outcome, rather than a limitation of the method. Since both algorithms frequently operate close to the lower bound, the room for further improvement is naturally restricted.

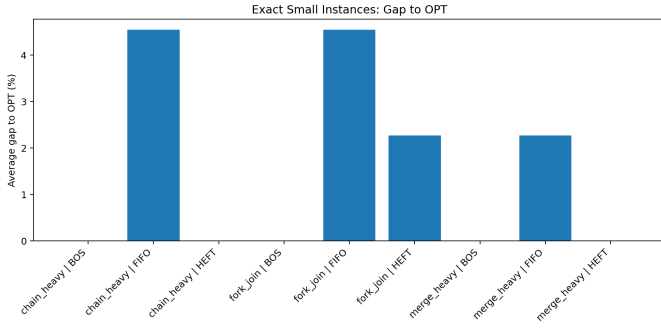


Fig. 5. Average gap to the optimal solution on small exact instances. BOS attains zero optimality gap on all tested families, while FIFO remains clearly suboptimal. This supports the validity of BOS beyond heuristic-level performance.

Second, BOS differs from the HEFT baseline in its scheduling logic. Both use static per-task ranks for a fixed instance, but BOS computes its rank from edge-wise overlap lags and applies it to an event-driven executable set. The observed disagreement rates show that this difference changes dispatch decisions.

Third, BOS remains clearly stronger than FIFO and reaches optimal performance on the tested exact small instances. This shows that BOS is not only theoretically motivated, but also empirically sound.

Overall, the contribution of BOS should therefore not be framed as “consistently outperforming HEFT by a large margin.” A more accurate interpretation is that BOS provides an alternative overlap-aware and event-driven bottleneck scheduling framework that remains competitive with HEFT, while being grounded in a different decision principle and supported by both lower-bound analysis and exact validation.

IX. CONCLUSION

This paper introduced BOS (Bottleneck-Oriented Overlap Scheduling), an event-driven scheduling framework for identical parallel machines with overlap-aware precedence constraints.

BOS computes a provably correct overlap-adjusted bottom level and applies that fixed score to the executable tasks exposed by release, activation, and completion events. A schedule-induced augmented DAG then separates precedence delays from machine-order delays.

Extensive experiments demonstrate that BOS achieves makespan performance comparable to HEFT across a wide range of benchmark instances. While BOS does not consistently outperform HEFT in absolute makespan, both methods are observed to operate close to the theoretical lower bound, indicating that the scheduling space is inherently tight.

More importantly, BOS exhibits substantial decision-process divergence from HEFT, as reflected by high disagreement rates across multiple instance groups. Despite these differences, both approaches converge to near-optimal solutions, suggesting the existence of multiple effective scheduling mechanisms under overlap-aware constraints.

In addition, BOS consistently outperforms FIFO and achieves optimal solutions in all tested small instances, further validating its effectiveness.

The complete analysis proves feasibility, termination, and near-linear event-driven implementation. It also establishes a $(2 - 1/m)$ approximation when all releases are zero and a safe 2-approximation for arbitrary release times. These bounds hold for the underlying work-conserving class; the BOS priority’s contribution is its interpretable dispatching behavior and empirical performance.

Future Work

Future work includes characterizing instance classes where the overlap-adjusted priority improves on alternative list orders, integrating the augmented-DAG diagnosis into online decisions, and extending BOS to heterogeneous machines, setup times, and stochastic scheduling.

APPENDIX A

EVENT SEMANTICS AND ADDITIONAL PROPERTIES

Although start times may be real-valued, the scheduler changes state only at the finite ordered event times $\tau_1 < \dots < \tau_K$. For a task that starts at s_i , define its processed amount at time t as

$$x_i(t) = \min\{p_i, \max\{0, t - s_i\}\}. \quad (32)$$

An edge (i, j) is activated at an event time t exactly when $x_i(t) \geq p_i - \delta_{ij} = \ell_{ij}$. Because a not-started predecessor has no recorded start event, the implementation also requires every predecessor to belong to $A(t) \cup C(t)$, as stated in (13).

Lemma 11 (Equivalence of progress and lag activation). *For a started predecessor i , the condition $x_i(t) \geq p_i - \delta_{ij}$ is equivalent to $t \geq s_i + \ell_{ij}$.*

Proof. Since $0 \leq \ell_{ij} \leq p_i$, task i has processed at least ℓ_{ij} units at time t exactly when at least ℓ_{ij} time units have elapsed since s_i . This is $t - s_i \geq \ell_{ij}$, or $t \geq s_i + \ell_{ij}$. $\square$

Lemma 12 (Monotonicity in overlap). *If $\delta'_{ij} \geq \delta_{ij}$ for every edge, then $Q_{\delta'}(P) \leq Q_{\delta}(P)$ for every path P , and hence $L_{\delta'} \leq L_{\delta}$.*

Proof. Using the second expression in (7), increasing any overlap allowance only increases a subtracted term. The inequality holds path by path and therefore also after taking the maximum. $\square$

When every $\delta_{ij} = 0$, (7) becomes the ordinary sum of processing times on a precedence path, and L_{δ} is the classical critical-path length. Overlap can reduce precedence waiting, but it never reduces W or creates additional machine capacity.

REFERENCES

- [1] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*. Springer, 2008.
- [2] G. W. Plossl and J. Orlicky, *Orlicky’s Material Requirements Planning*. McGraw-Hill, 1994.
- [3] D. D. Yao, “Book review: Stochastic models of manufacturing systems,” *Discrete Event Dynamic Systems*, vol. 4, no. 4, pp. 409–411, 1994.

- [4] J. Adams, E. Balas, and D. Zawack, "The shifting bottleneck procedure for job shop scheduling," *Management Science*, vol. 34, no. 3, pp. 391–401, 1988.
- [5] R. W. Conway, W. L. Maxwell, and L. W. Miller, *Theory of Scheduling*. Addison-Wesley, 1967.
- [6] M. R. Garey and D. S. Johnson, *Computers and Intractability*. W. H. Freeman, 1979.
- [7] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [8] P. Brucker, A. Drexler, R. H. Möhring, K. Neumann, and E. Pesch, "Resource-constrained project scheduling," *European Journal of Operational Research*, vol. 112, no. 1, pp. 3–41, 1999.
- [9] R. Kolisch and S. Hartmann, "Heuristic algorithms for RCPSP," in *Project Scheduling*. Springer, 1999, pp. 147–178.
- [10] W. Herroelen and R. Leus, "Project scheduling under uncertainty: Survey and research potentials," *European Journal of Operational Research*, vol. 165, no. 2, pp. 289–306, 2005.
- [11] L. P. Leach, *Critical Chain Project Management*. Artech House, 1999.
- [12] L. Lawrence, *Critical Chain Project Management*. Artech House, 2014.
- [13] D. B. Shmoys, C. Stein, and J. Wein, "Improved approximation algorithms for shop scheduling," in *Proceedings of the Second Annual ACM-SIAM Symposium on Discrete Algorithms*, 1991.
- [14] O. Svensson, "Conditional hardness of precedence constrained scheduling on identical machines," in *Proceedings of the 42nd ACM Symposium on Theory of Computing*, 2010, pp. 745–754.
- [15] C. N. Potts and M. Y. Kovalyov, "Scheduling with batching: A review," *European Journal of Operational Research*, vol. 120, no. 2, pp. 228–249, 2000.
- [16] R. H. Möhring, M. Skutella, and F. Stork, "Scheduling with AND/OR precedence constraints," *SIAM Journal on Computing*, vol. 33, no. 2, pp. 393–415, 2004.
- [17] M. Gómez Sánchez, E. Lalla-Ruiz, A. Fernández Gil, C. Castro, and S. Voß, "Resource-constrained multi-project scheduling problem: A survey," *European Journal of Operational Research*, vol. 309, no. 3, pp. 958–976, 2023.
- [18] M. Aghileh, A. Tereso, F. Alvelos, and M. O. M. Lopes, "Multi-project scheduling with uncertainty and resource flexibility," *Algorithms*, vol. 18, no. 6, p. 314, 2025.
- [19] M. Urgo, G. Lanza, R. Vrabich, and D. Gyulai, "Future-proof production scheduling and control," *CIRP Annals*, vol. 74, no. 2, pp. 971–991, 2025.
- [20] M. E. Kanakis, R. Khalili, and L. Wang, "Machine learning for computer systems and networking: A survey," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–36, 2022.
- [21] J. Orlicky, *Material Requirements Planning*. McGraw-Hill, 1975.
- [22] T. Brockhoff, P. Mavrothalassitis, E. Bakopoulos, N. Nikolakis, and D. Mourtzis, "Analyzing multi-level BOM-structured event data," in *Process Mining Workshops*, 2022, pp. 47–59.
- [23] W. J. Hopp and M. L. Spearman, *Factory Physics*. Waveland Press, 2011.
- [24] C. Roser, M. Nakano, and M. Tanaka, "Shifting bottleneck detection," in *Proceedings*, 2003.
- [25] J. B. Kidd and Y. Monden, "Toyota production system," *Journal of the Operational Research Society*, vol. 46, p. 669, 1993.
- [26] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [27] L. A. Hall, "Approximability of flow shop scheduling," *Mathematical Programming*, vol. 82, no. 1, pp. 175–190, 1998.
- [28] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
- [29] C. N. Potts and L. N. Van Wassenhove, "Integrating scheduling and batching decisions for manufacturing systems," *European Journal of Operational Research*, vol. 56, no. 2, pp. 204–217, 1992.
- [30] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of HotNets*, 2016.
- [31] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proceedings of SIGCOMM*, 2019.
- [32] K. Alexopoulos *et al.*, "Deep reinforcement learning for selection of dispatch rules," *Applied Sciences*, vol. 15, no. 1, p. 232, 2025.